



Outline

- ✓ Importing a module
- ✓ Series in Pandas
- ✓ Creating Time Series
- ✓ Data Frames
- ✓ Conditional Selection
- ✓ Read CSV
- ✓ Read Excel
- ✓ Setting/Resetting index
- ✓ Reading in Multiindexed DataFrame
- ✓ directly from CSV
- ✓ Cross Section
- ✓ Dealing with Missing Data
- ✓ Groupby
- ✓ Concatenation, Join, Merge

Series

- ▶ Series is an one-dimensional* array with axis labels.
- ▶ It supports both integer and label-based index but index must be of hashable type.
- ▶ If we do not specify index it will assign integer zero-based index.

syntax

```
import pandas as pd  
s = pd.Series(data, index, dtype, copy=False)
```

Parameters

data = array like Iterable
index = array like index
dtype = data-type
copy = bool, default is False

pandasSeries.py

```
1 import pandas as pd  
2 s = pd.Series([1, 3, 5, 7, 9, 11])  
3 print(s)
```

Output

```
0    1  
1    3  
2    5  
3    7  
4    9  
5   11  
dtype: int64
```

Series (Cont.)

- ▶ We can then access the elements inside Series just like array using square brackets notation.

pdSeriesEle.py

```
1 import pandas as pd
2 s = pd.Series([1, 3, 5, 7, 9, 11])
3 print("S[0] = ", s[0])
4 b = s[0] + s[1]
5 print("Sum = ", b)
```

Output

```
S[0] = 1
Sum = 4
```

- ▶ We can specify the data type of Series using **dtype** parameter

pdSeriesdtype.py

```
1 import pandas as pd
2 s = pd.Series([1, 3, 5, 7, 9, 11], dtype='str')
3 print("S[0] = ", s[0])
4 b = s[0] + s[1]
5 print("Sum = ", b)
```

Output

```
S[0] = 1
Sum = 13
```

Series (Cont.)

- ▶ We can specify index to Series with the help of **index** parameter

pdSeriesdtype.py

```
1 import numpy as np
2 import pandas as pd
3 i = ['name', 'address', 'phone', 'email', 'website']
4 d = ['darshan', 'rj', 123, 'd@d.com', 'darshan.ac.in']
5 s = pd.Series(data=d, index=i)
6 print(s)
```

Output

```
name          darshan
address       rj
phone         123
email         d@d.com
website      darshan.ac.in
dtype: object
```

Creating Time Series

- ▶ We can use some of pandas inbuilt date functions to create a time series.

pdSeriesEle.py

```
1 import numpy as np
2 import pandas as pd
3 dates = pd.to_datetime("27th of July, 2020")
4 i = dates + pd.to_timedelta(np.arange(5),
5                             unit='D')
5 d = [50,53,25,70,60]
6 time_series = pd.Series(data=d,index=i)
7 print(time_series)
```

Output

2020-07-27	50
2020-07-28	53
2020-07-29	25
2020-07-30	70
2020-07-31	60
dtype: int64	

Data Frames

- ▶ Data frames are two dimensional data structure, i.e. data is aligned in a tabular format in rows and columns.
- ▶ Data frame also contains labelled axes on rows and columns.
- ▶ Features of Data Frame :
 - ➔ It is size-mutable
 - ➔ Has labelled axes
 - ➔ Columns can be of different data types
 - ➔ We can perform arithmetic operations on rows and columns.

- ▶ Structure :

	PDS	Algo	SE	INS
101				
102				
103				
....				
160				

Data Frames (Cont.)

► Syntax :

syntax

```
import pandas as pd
df = pd.DataFrame(data, index, columns, dtype, copy=False)
```

Parameters

data = array like Iterable
index = array like row index
columns = array like col index
dtype = data-type
copy = bool, default is False

► Example :

pdDataFrame.py

```
1 import numpy as np
2 import pandas as pd
3 randArr = np.random.randint(0,100,20).reshape(5,4)
4 df =
  pd.DataFrame(randArr,np.arange(101,106,1), ['PDS', 'Algo', 'SE', 'INS'])
5 print(df)
```

Output

	PDS	Algo	SE	INS
101	0	23	93	46
102	85	47	31	12
103	35	34	6	89
104	66	83	70	50
105	65	88	87	87

Data Frames (Cont.)

- Grabbing the column

dfGrabCol.py

```
1 import numpy as np
2 import pandas as pd
3 randArr = np.random.randint(0,100,20).reshape(5,4)
4 df =
  pd.DataFrame(randArr,np.arange(101,106,1),[ 'PDS', 'A
  lgo', 'SE', 'INS'])
5 print(df[ 'PDS' ])
```

Output

101	0
102	85
103	35
104	66
105	65

Name: PDS, dtype: int32

- Grabbing the multiple column

dfGrabMulCol.py

```
1 print(df[[ 'PDS', 'SE' ]])
```

Output

	PDS	SE
101	0	93
102	85	31
103	35	6
104	66	70
105	65	87

Data Frames (Cont.)

▶ Grabbing a row

dfGrabRow.py

```
1 print(df.loc[101]) # using labels
2 #OR
3 print(df.iloc[0]) # using zero based index
```

Output

PDS	0
Algo	23
SE	93
INS	46
Name: 101, dtype: int32	

▶ Grabbing Single Value

dfGrabSingle.py

```
1 print(df.loc[101, 'PDS']) # using labels
```

Output

0

▶ Deleting Row

dfDelCol.py

```
1 df.drop('103',inplace=True)
2 print(df)
```

Output

	PDS	Algo	SE	INS
101	0	23	93	46
102	85	47	31	12
104	66	83	70	50
105	65	88	87	87

Data Frames (Cont.)

▶ Creating new column

dfCreateCol.py

```
1 df['total'] = df['PDS'] + df['Algo'] +  
  df['SE'] + df['INS']  
2 print(df)
```

Output

	PDS	Algo	SE	INS	total
101	0	23	93	46	162
102	85	47	31	12	175
103	35	34	6	89	164
104	66	83	70	50	269
105	65	88	87	87	327

▶ Deleting Column and Row

dfDelCol.py

```
1 df.drop('total',axis=1,inplace=True)  
2 print(df)
```

Output

	PDS	Algo	SE	INS
101	0	23	93	46
102	85	47	31	12
103	35	34	6	89
104	66	83	70	50
105	65	88	87	87

Data Frames (Cont.)

▶ Getting Subset of Data Frame

dfGrabSubSet.py

```
1 print(df.loc[[101,104], [['PDS', 'INS']]])
```

Output

	PDS	INS
101	0	46
104	66	50

▶ Selecting all cols except one

dfGrabExcept.py

```
1 print(df.loc[:, df.columns != 'Algo' ])
```

Output

	PDS	SE	INS
101	0	93	46
102	85	31	12
103	35	6	89
104	66	70	50
105	65	87	87

Conditional Selection

- ▶ Similar to NumPy we can do conditional selection in pandas.

dfCondSel.py

```
1 import numpy as np
2 import pandas as pd
3 np.random.seed(121)
4 randArr =
  np.random.randint(0,100,20).reshape(5,4)
5 df =
  pd.DataFrame(randArr,np.arange(101,106,1)
  ,['PDS','Algo','SE','INS'])
6 print(df)
7 print(df>50)
```

Output

	PDS	Algo	SE	INS
101	66	85	8	95
102	65	52	83	96
103	46	34	52	60
104	54	3	94	52
105	57	75	88	39
	PDS	Algo	SE	INS
101	True	True	False	True
102	True	True	True	True
103	False	False	True	True
104	True	False	True	True
105	True	True	True	False

- ▶ Note : we have used np.random.seed() method and set seed to be 121, so that when you generate random number it matches with the random number I have generated.

Conditional Selection (Cont.)

- ▶ We can then use this boolean DataFrame to get associated values.

dfCondSel.py

```
1 dfBool = df > 50
2 print(df[dfBool])
```

- ▶ Note : It will set NaN (Not a Number) in case of False

Output

	PDS	Algo	SE	INS
101	66	85	NaN	95
102	65	52	83	96
103	NaN	NaN	52	60
104	54	NaN	94	52
105	57	75	88	NaN

- ▶ We can apply condition on specific column.

dfCondSel.py

```
1 dfBool = df['PDS'] > 50
2 print(df[dfBool])
```

Output

	PDS	Algo	SE	INS
101	66	85	8	95
102	65	52	83	96
104	54	3	94	52
105	57	75	88	39

Setting/Resetting index

- ▶ In our previous example we have seen our index does not have name, if we want to specify name to our index we can specify it using **DataFrame.index.name** property.

dfCondSel.py

```
1 df.index.name = 'RollNo'
```

Note: We have name to our index now

Output

	PDS	Algo	SE	INS
RollNo				
101	66	85	8	95
102	65	52	83	96
103	46	34	52	60
104	54	3	94	52
105	57	75	88	39

- ▶ We can use pandas built-in methods to set or reset the index
 - ➔ `pd.set_index('NewColumn',inplace=True)`, will set new column as index,
 - ➔ `pd.reset_index()`, will reset index to zero based numeric index.

Setting/Resetting index (Cont.)

► set_index(new_index)

dfCondSel.py

```
1 df.set_index('PDS') #inplace=True
```

Note: We have PDS as our index now

Output

	Algo	SE	INS
PDS			
66	85	8	95
65	52	83	96
46	34	52	60
54	3	94	52
57	75	88	39

► reset_index()

dfCondSel.py

```
1 df.reset_index() #inplace=True
```

Note: Our **RollNo(index)** become new column, and we now have zero based numeric index

Output

	RollNo	PDS	Algo	SE	INS
0	101	66	85	8	95
1	102	65	52	83	96
2	103	46	34	52	60
3	104	54	3	94	52
4	105	57	75	88	39

Multi-Index DataFrame

- ▶ Hierarchical indexes (AKA multiindexes) help us to organize, find, and aggregate information faster at almost no cost.
- ▶ Example where we need Hierarchical indexes

Numeric Index/Single Index

	Col	Dep	Sem	RN	S1	S2	S3
0	ABC	CE	5	101	50	60	70
1	ABC	CE	5	102	48	70	25
2	ABC	CE	7	101	58	59	51
3	ABC	ME	5	101	30	35	39
4	ABC	ME	5	102	50	90	48
5	Darshan	CE	5	101	88	99	77
6	Darshan	CE	5	102	99	84	76
7	Darshan	CE	7	101	88	77	99
8	Darshan	ME	5	101	44	88	99

Multi Index

			RN	S1	S2	S3
Col	Dep	Sem				
ABC	CE	5	101	50	60	70
		5	102	48	70	25
		7	101	58	59	51
	ME	5	101	30	35	39
Darshan	CE	5	102	50	90	48
		5	101	88	99	77
		5	102	99	84	76
		7	101	88	77	99
	ME	5	101	44	88	99

Multi-Index DataFrame (Cont.)

- ▶ Creating multiindexes is as simple as creating single index using `set_index` method, only difference is in case of multiindexes we need to provide list of indexes instead of a single string index, lets see an example for that

dfMultiIndex.py

```
1 dfMulti =  
  pd.read_csv('MultiIndexDemo.csv')  
2 dfMulti.set_index(['Col', 'Dep', 'Sem'],  
  inplace=True)  
3 print(dfMulti)
```

Output

			RN	S1	S2	S3
Col	Dep	Sem				
ABC	CE	5	101	50	60	70
		5	102	48	70	25
		7	101	58	59	51
	ME	5	101	30	35	39
		5	102	50	90	48
		7	101	88	77	99
Darshan	CE	5	101	88	99	77
		5	102	99	84	76
		7	101	88	77	99
	ME	5	101	44	88	99

Multi-Index DataFrame (Cont.)

► Now we have multi-indexed DataFrame from which we can access data using multiple index

► For Example

→ Sub DataFrame for all the students of Darshan

dfGrabDarshanStu.py

```
1 print(dfMulti.loc['Darshan'])
```

→ Sub DataFrame for Computer Engineering students from Darshan

dfGrabDarshanCEStu.py

```
1 print(dfMulti.loc['Darshan', 'CE'])
```

Output (Darshan)

		RN	S1	S2	S3
Dep	Sem				
CE	5	101	88	99	77
	5	102	99	84	76
	7	101	88	77	99
ME	5	101	44	88	99

Output (Darshan->CE)

		RN	S1	S2	S3
Sem					
5		101	88	99	77
5		102	99	84	76
7		101	88	77	99

Reading in Multiindexed DataFrame directly from CSV

- ▶ `read_csv` function of pandas provides easy way to create multi-indexed DataFrame directly while fetching the CSV file.

dfMultiIndex.py

```
1 dfMultiCSV =  
  pd.read_csv('MultiIndexDemo.csv',  
             ,index_col=[0,1,2])  
  #for multi-index in cols we can  
  use header parameter  
2 print(dfMultiCSV)
```

Output

			RN	S1	S2	S3
Col	Dep	Sem				
ABC	CE	5	101	50	60	70
		5	102	48	70	25
		7	101	58	59	51
	ME	5	101	30	35	39
		5	102	50	90	48
		7	101	88	77	99
Darshan	CE	5	101	88	99	77
		5	102	99	84	76
		7	101	88	77	99
	ME	5	101	44	88	99

Cross Sections in DataFrame

- ▶ The `xs()` function is used to get cross-section from the Series/DataFrame.
- ▶ This method takes a key argument to select data at a particular level of a MultiIndex.

- ▶ Syntax :

syntax

```
DataFrame.xs(key, axis=0, level=None, drop_level=True)
```

- ▶ Example :

dfMultiIndex.py

```
1 dfMultiCSV =  
  pd.read_csv('MultiIndexDemo.csv',  
    index_col=[0,1,2])  
2 print(dfMultiCSV)  
3 print(dfMultiCSV.xs('CE',axis=0,level='Dep'))
```

=== Parameters ===

key : label

axis : Axis to retrieve
cross section

level : level of key

drop_level : False if you
want to preserve the level

Output

RN	S1	S2	S3			
Col	Sem					
ABC	5	101	50	60	70	
	5	102	48	70	25	
	7	101	58	59	51	
Darshan	5	101	88	99	77	
	5	102	99	84	76	
	7	101	88	77	99	
	5	102	99	84	76	
	7	101	88	77	99	
ME	5	101	44	88	99	

Dealing with Missing Data

- ▶ There are many methods by which we can deal with the missing data, some of most commons are listed below,
 - ➔ dropna, will drop (delete) the missing data (rows/cols)
 - ➔ fillna, will fill specified values in place of missing data
 - ➔ interpolate, will interpolate missing data and fill interpolated value in place of missing data.

- ▶ it will drop(delete) the missing data(rows/cols)

syntax

```
DataFrame.dropna(axis, how, inplace)
```

Parameters	Description
axis	Determine if rows or columns which contain missing values are removed.
how	Determine if a row or column is removed from DataFrame when we have at least one NA or all NA. • 'any': If any NA values are present, drop that row or column. • 'all': If all values are NA, drop that row or column.
inplace	If True, do the operation in place and return None.

dropna - Example

demo.py

```
1 import pandas as pd
2 import numpy as np
3 randArr = np.array([np.nan,2,3,4,6,7,8,9,]).reshape(2,4)
4 df = pd.DataFrame(randArr,np.arange(101,103,1),['PDS','Algo','SE','INS'])
5 print(df)
6 df.dropna()
```

Output

	PDS	Algo	SE	INS
101	NaN	2.0	3.0	4.0
102	6.0	7.0	8.0	9.0

	PDS	Algo	SE	INS
102	6.0	7.0	8.0	9.0

- ▶ It will fill specified values in place of missing data.

syntax

```
DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None, downcast=None)
```

Parameters	Description
value	Value to use to fill holes (e.g. 0), alternately a dict/Series/DataFrame of values specifying which value to use for each index (for a Series) or column (for a DataFrame). Values not in the dict/Series/DataFrame will not be filled. This value cannot be a list.
method	Method to use for filling holes in reindexed Series pad / ffill: propagate last valid observation forward to next valid backfill / bfill: use next valid observation to fill gap.
axis	Axis along which to fill missing values.
inplace	If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).
limit	If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill.
downcast	A dict of item->dtype of what to downcast if possible, or the string 'infer' which will try to downcast to an appropriate equal type

fillna- Example

demo.py

```
1 df = pd.DataFrame([[np.nan, 2, np.nan, 0],[3, 4, np.nan, 1],  
2 [np.nan, np.nan, np.nan, np.nan],[np.nan, 3, np.nan, 4]],  
3 columns=list("ABCD"))  
4  
5 df.fillna(method='bfill')
```

Output

	A	B	C	D
0	3.0	2.0	NaN	0.0
1	3.0	4.0	NaN	1.0
2	NaN	3.0	NaN	4.0
3	NaN	3.0	NaN	4.0

interpolate

- ▶ it will interpolate missing data and fill interpolated values in place of missing data.

syntax

```
DataFrame.interpolate(method='linear', axis=0, limit=None, inplace=False, limit_direction='forward', limit_area=None, downcast=None)
```

Parameters	Description
method	Interpolation technique to use. One of: Linear, time, index, pad, nearest, zero, slinear, quadratic, cubic, spline, barycentric, krogh, from_derivatives.
axis	Axis to interpolate along.
inplace	Update the data in place if possible.
limit	The maximum number of consecutive NaNs to fill. Must be greater than 0.
limit_direction	If the limit is specified, consecutive NaNs will be filled in this direction.
limit_area	If the limit is specified, consecutive NaNs will be filled with this restriction.
downcast	Downcast dtypes if possible.

interpolate Example

demo.py

```
1 df = pd.DataFrame([(0.0, np.nan, -1.0, 1.0), (np.nan, 2.0, np.nan,  
2 np.nan), (2.0, 3.0, np.nan, 9.0), (np.nan, 4.0, -4.0, 16.0)],  
3 columns=list('abcd'))  
4  
5 df.interpolate(method='linear', limit_direction='forward', axis=0)
```

Output

	a	b	c	d
0	0.0	NaN	-1.0	1.0
1	1.0	2.0	-2.0	5.0
2	2.0	3.0	-3.0	9.0
3	2.0	4.0	-4.0	16.0

Groupby in Pandas

- ▶ Any groupby operation involves one of the following operations on the original object. They are
 - ➔ **Splitting** the Object
 - ➔ **Applying** a function
 - ➔ **Combining** the results
- ▶ In many situations, we split the data into sets and we apply some functionality on each subset.
- ▶ we can perform the following operations
 - ➔ **Aggregation** – computing a summary statistic
 - ➔ **Transformation** – perform some group-specific operation
 - ➔ **Filtration** – discarding the data with some condition
- ▶ Basic ways to use of groupby method
 - ➔ `df.groupby('key')`
 - ➔ `df.groupby(['key1', 'key2'])`
 - ➔ `df.groupby(key, axis=1)`

College	Enno	CPI
Darshan	123	8.9
Darshan	124	9.2
Darshan	125	7.8
Darshan	128	8.7
ABC	211	5.6
ABC	212	6.2
ABC	215	3.2
ABC	218	4.2
XYZ	312	5.2
XYZ	315	6.5
XYZ	315	5.8

College	Mean CPI
Darshan	8.65
ABC	4.8
XYZ	5.83

Groupby in Pandas (Cont.)

► Example : Listing all the groups

dfGroup.py

```
1 dfIPL = pd.read_csv('IPLDataSet.csv')
2 print(dfIPL.groupby('Year').groups)
```

Output

```
{2014: Int64Index([0, 2, 4, 9],
dtype='int64'),
2015: Int64Index([1, 3, 5, 10],
dtype='int64'),
2016: Int64Index([6, 8],
dtype='int64'),
2017: Int64Index([7, 11],
dtype='int64')}
```

Groupby in Pandas (Cont.)

► Example : Group by multiple columns

dfGroupMul.py

```
1 dfIPL = pd.read_csv('IPLDataSet.csv')
2 print(dfIPL.groupby(['Year','Team']).groups)
```

Output

```
{(2014, 'Devils'): Int64Index([2],
dtype='int64'),
 (2014, 'Kings'): Int64Index([4],
dtype='int64'),
 (2014, 'Riders'): Int64Index([0],
dtype='int64'),
 .....
 .....
 (2016, 'Riders'): Int64Index([8],
dtype='int64'),
 (2017, 'Kings'): Int64Index([7],
dtype='int64'),
 (2017, 'Riders'): Int64Index([11],
dtype='int64')}
```

Groupby in Pandas (Cont.)

► Example : Iterating through groups

dfGrouplter.py

```
1 dfIPL = pd.read_csv('IPLDataSet.csv')
2 groupIPL = dfIPL.groupby('Year')
3 for name,group in groupIPL :
4     print(name)
5     print(group)
```

Output

2014

	Team	Rank	Year	Points
0	Riders	1	2014	876
2	Devils	2	2014	863
4	Kings	3	2014	741
9	Royals	4	2014	701

2015

	Team	Rank	Year	Points
1	Riders	2	2015	789
3	Devils	3	2015	673
5	kings	4	2015	812
10	Royals	1	2015	804

2016

	Team	Rank	Year	Points
6	Kings	1	2016	756
8	Riders	2	2016	694

2017

	Team	Rank	Year	Points
7	Kings	1	2017	788
11	Riders	2	2017	690

Groupby in Pandas (Cont.)

► Example : Aggregating groups

dfGroupAgg.py

```
1 dfSales = pd.read_csv('SalesDataSet.csv')
2 print(dfSales.groupby(['YEAR_ID']).count(
3     )['QUANTITYORDERED'])
4 print(dfSales.groupby(['YEAR_ID']).sum()['QUANTITYORDERED'])
5 print(dfSales.groupby(['YEAR_ID']).mean()['QUANTITYORDERED'])
```

Output

```
YEAR_ID
2003      1000
2004      1345
2005       478
Name: QUANTITYORDERED, dtype:
int64
YEAR_ID
2003      34612
2004      46824
2005      17631
Name: QUANTITYORDERED, dtype:
int64
YEAR_ID
2003      34.612000
2004      34.813383
2005      36.884937
Name: QUANTITYORDERED, dtype:
float64
```


Groupby in Pandas (Cont.)

► Example : Describe details

dfGroupDesc.py

```
1 dfIPL =  
  pd.read_csv('IPLDataSet.csv')  
2 print(dfIPL.groupby('Year').describe()[  
  'Points'])
```

Output

	count	mean	std	min
	25%	50%	75%	max
Year				
2014	4.0	795.25	87.439026	701.0
	731.0	802.0	866.25	876.0
2015	4.0	769.50	65.035888	673.0
	760.0	796.5	806.00	812.0
2016	2.0	725.00	43.840620	694.0
	709.5	725.0	740.50	756.0
2017	2.0	739.00	69.296465	690.0
	714.5	739.0	763.50	788.0

Concatenation in Pandas

- ▶ Concatenation basically glues together DataFrames.
- ▶ Keep in mind that dimensions should match along the axis you are concatenating on.
- ▶ You can use **pd.concat** and pass in a list of DataFrames to concatenate together:

dfConcat.py

```
1 dfCX = pd.read_csv('CX_Marks.csv',index_col=0)
2 dfCY = pd.read_csv('CY_Marks.csv',index_col=0)
3 dfCZ = pd.read_csv('CZ_Marks.csv',index_col=0)
4 dfAllStudent = pd.concat([dfCX,dfCY,dfCZ])
5 print(dfAllStudent)
```

Output

	PDS	Algo	SE
101	50	55	60
102	70	80	61
103	55	89	70
104	58	96	85
201	77	96	63
202	44	78	32
203	55	85	21
204	69	66	54
301	11	75	88
302	22	48	77
303	33	59	68
304	44	55	62

- ▶ Note : We can use **axis=1** parameter to concat columns.

Join in Pandas

- ▶ `df.join()` method will efficiently join multiple DataFrame objects by **index**(or column specified).
- ▶ some of important **Parameters** :
 - ➔ **dfOther** : Right Data Frame
 - ➔ **on** (Not recommended) : specify the column on which we want to join (Default is index)
 - ➔ **how** : How to handle the operation of the two objects.
 - **left**: use calling frame's index (*Default*).
 - **right**: use dfOther index.
 - **outer**: form union of calling frame's index with other's index (or column if on is specified), and sort it. lexicographically.
 - **inner**: form intersection of calling frame's index (or column if on is specified) with other's index, preserving the order of the calling's one.

Join in Pandas (Example)

dfJoin.py

```
1 dfINS =  
  pd.read_csv('INS_Marks.csv', index_col=0)  
2 dfLeftJoin = allStudent.join(dfINS)  
3 print(dfLeftJoin)  
4 dfRightJoin =  
  allStudent.join(dfINS, how='right')  
5 print(dfRightJoin)
```

Output - 1

	PDS	Algo	SE	INS
101	50	55	60	55.0
102	70	80	61	66.0
103	55	89	70	77.0
104	58	96	85	88.0
201	77	96	63	66.0
202	44	78	32	NaN
203	55	85	21	78.0
204	69	66	54	85.0
301	11	75	88	11.0
302	22	48	77	22.0
303	33	59	68	33.0
304	44	55	62	44.0

Output - 2

	PDS	Algo	SE	INS
301	11	75	88	11
302	22	48	77	22
303	33	59	68	33
304	44	55	62	44
101	50	55	60	55
102	70	80	61	66
103	55	89	70	77
104	58	96	85	88
201	77	96	63	66
203	55	85	21	78
204	69	66	54	85

Merge in Pandas

- ▶ Merge DataFrame or named Series objects with a database-style join.
- ▶ Similar to join method, but used when we want to join/merge with the columns instead of index.
- ▶ some of important **Parameters** :
 - ➔ **dfOther** : Right Data Frame
 - ➔ **on** : specify the column on which we want to join (Default is index)
 - ➔ **left_on** : specify the column of left Dataframe
 - ➔ **right_on** : specify the column of right Dataframe
 - ➔ **how** : How to handle the operation of the two objects.
 - **left**: use calling frame's index (*Default*).
 - **right**: use dfOther index.
 - **outer**: form union of calling frame's index with other's index (or column if on is specified), and sort it. lexicographically.
 - **inner**: form intersection of calling frame's index (or column if on is specified) with other's index, preserving the order of the calling's one.

Merge in Pandas (Example)

dfMerge.py

```
1 m1 = pd.read_csv('Merge1.csv')
2 print(m1)
3 m2 = pd.read_csv('Merge2.csv')
4 print(m2)
5 m3 = m1.merge(m2,on='EnNo')
6 print(m3)
```

Output

	RollNo	EnNo	Name
0	101	11112222	Abc
1	102	11113333	Xyz
2	103	22224444	Def

	EnNo	PDS	INS
0	11112222	50	60
1	11113333	60	70

	RollNo	EnNo	Name	PDS	INS
0	101	11112222	Abc	50	60
1	102	11113333	Xyz	60	70

Read CSV in Pandas

- ▶ **read_csv()** is used to read Comma Separated Values (CSV) file into a pandas DataFrame.
- ▶ some of important **Parameters** :
 - ➔ **filePath** : str, path object, or file-like object
 - ➔ **sep** : separator (Default is comma)
 - ➔ **header**: Row number(s) to use as the column names.
 - ➔ **index_col** : index column(s) of the data frame.

readCSV.py

```
1 dfINS = pd.read_csv('Marks.csv', index_col=0, header=0)
2 print(dfINS)
```

Output

	PDS	Algo	SE	INS
101	50	55	60	55.0
102	70	80	61	66.0
103	55	89	70	77.0
104	58	96	85	88.0
201	77	96	63	66.0

Read Excel in Pandas

- ▶ Read an Excel file into a pandas DataFrame.
- ▶ Supports *xls*, *xlsx*, *xlsm*, *xlsb*, *odf*, *ods* and *odt* file extensions read from a local filesystem or URL. Supports an option to read a single sheet or a list of sheets.
- ▶ some of important **Parameters** :
 - ➔ **excelFile** : str, bytes, ExcelFile, xlrd.Book, path object, or file-like object
 - ➔ **sheet_name** : sheet no in integer or the name of the sheet, can have list of sheets.
 - ➔ **index_col** : index column of the data frame.